

Programmieren für Kids

JavaScript für Kids

Modul 7

Steuern mit der Tastatur



Trainer: Wolfgang Schedl

Kurz zurückgeblickt

Bisher steuerst du deine Spiele mit der **Maus**: klicken, klicken, klicken. Echte Spiele steuert man aber mit der **Tastatur**. Heute bewegst du eine Figur mit den **Pfeiltasten** über den Bildschirm.

Das baust du in diesem Modul

- eine **Figur**, die auf die Pfeiltasten hört und sich bewegt
- ein **Fang-Spiel**: sammle die Sterne ein, so schnell du kannst

Ab jetzt fühlt es sich wie ein echtes Computerspiel an.



Kapitel 1

Steuern mit der Tastatur

⚡ Was ist ein Event?

Ein **Event** ist „etwas passiert“, worauf deine Seite reagieren kann:

- ein Klick auf einen Knopf (kennst du schon)
- eine gedrückte Taste (das lernst du jetzt)
- die Maus bewegt sich, ein Timer tickt ...

Für jedes Event kannst du sagen: „Wenn das passiert, ruf diese Funktion.“



Ein neues Event: keydown

Ein Klick war ein **Event**: etwas passiert, und eine Funktion reagiert. Eine gedrückte Taste ist auch ein Event:

```
document.addEventListener("keydown", bewege)

function bewege(event) {
  console.log("Eine Taste wurde gedrückt!")
}
```

addEventListener sagt: „Wenn eine Taste gedrückt wird, ruf bewege auf.“



Welche Taste? event.key

Die Funktion bekommt ein event. Darin steht, **welche** Taste gedrückt wurde:

```
function bewege(event) {  
  console.log(event.key)  // z.B. "ArrowRight"  
}
```

Die Pfeiltasten heißen "ArrowUp", "ArrowDown", "ArrowLeft" und "ArrowRight".

📌 Eine Figur hat einen Ort

Deine Figur steht an einer Stelle: x sagt, wie weit von links, y, wie weit von oben.

```
let x = 100  
let y = 100
```

Damit sich die Figur frei bewegen kann, hat sie im CSS `position: absolute` (bekommst du fertig).

Die Figur verschieben

Du verschiebst die Figur, indem du `left` (von links) und `top` (von oben) setzt:

```
document.getElementById("figur").style.left = x + "px"  
document.getElementById("figur").style.top = y + "px"
```

Vergiss das `"px"` nicht! Der Computer will wissen: 100 *Pixel*.



So ist deine Vorlage aufgebaut

```
<div id="figur">Held</div>

<style>
  #figur { position: absolute; }
</style>
```

Eine Figur namens **figur**. Das CSS mit `position: absolute` ist schon da, du schreibst nur das JavaScript.

Die Figur bewegen

```
let x = 100
let y = 100

document.addEventListener("keydown", bewege)

function bewege(event) {
  if (event.key === "ArrowRight") x = x + 20
  else if (event.key === "ArrowLeft") x = x - 20
  else if (event.key === "ArrowUp") y = y - 20
  else if (event.key === "ArrowDown") y = y + 20

  document.getElementById("figur").style.left = x + "px"
  document.getElementById("figur").style.top = y + "px"
}
```

Teste deine Figur

Speichern und ausprobieren, bevor du weiterbaust:

- Bewegt sich die Figur in alle vier Richtungen?
- Klick zuerst einmal auf die Seite, damit die Tasten wirken

Wenn nichts passiert: Steht `position: absolute` im CSS? Und hast du das `"px"` dabei?

[] Bleib auf dem Bildschirm

Ohne Grenzen läuft die Figur vom Bildschirm. Mit `if` halten wir sie drin:

```
if (x < 0) x = 0
if (x > 400) x = 400
if (y < 0) y = 0
if (y > 300) y = 300
```

Diese Zeilen kommen in `bewege`, direkt vor dem Anzeigen.

Die Taste genau schreiben

Der Name der Taste muss **exakt** stimmen, auch die Groß- und Kleinschreibung:

```
if (event.key === "ArrowRight") // richtig
if (event.key === "arrowright") // FALSCH
if (event.key === "Rechts")     // FALSCH
```

Schreib `console.log(event.key)`, wenn du unsicher bist, welche Taste welchen Namen hat.

Übung: bewege deine Figur



ÜBUNG

ca. 15 Minuten

- Bring die Figur mit allen vier Pfeiltasten zum Laufen
- Probiere verschiedene Schrittweiten (10, 20, 50)

★ **Bonus:** Lass die Figur auch auf die Buchstaben w a s d hören.

Fehler-Detektive



FINDE DEN FEHLER

Die Figur zählt zwar mit, bewegt sich aber nicht. Warum?

```
x = x + 20  
document.getElementById("figur").style.left = x
```

Tipp: Woher weiß der Computer, dass *x* in *Pixeln* gemeint ist?



Projekt

Fang den Stern



Jetzt öffnen: [7a-fang-den-stern](#)

in VS Code – oder auf der Kursseite

★ Wie es geht

- ein Stern steht irgendwo auf dem Bildschirm
- du bewegst deine Figur mit den Pfeiltasten dorthin
- berührst du den Stern, gibt es einen Punkt
- und der Stern springt an eine neue, zufällige Stelle

Alles Bausteine, die du kennst: Bewegen, Zufall, Entscheidung.



So ist deine Vorlage aufgebaut

```
<div id="figur">Held</div>  
<div id="stern">Stern</div>  
<h2 id="punkte">Punkte: 0</h2>
```

Deine **figur**, ein **stern** zum Einsammeln und eine Punkte-Anzeige. Beide haben `position: absolute`.

Das Ziel platzieren

Der Stern hat auch einen Ort. Den merken wir uns in `zielX` und `zielY`:

```
let zielX = 300  
let zielY = 150  
let punkte = 0
```

Am Anfang steht der Stern bei (300, 150). Gleich soll er zufällig springen.

🎯 Bist du nah dran?

Nach jeder Bewegung prüfen wir, ob die Figur nah genug am Stern ist:

```
function pruefe() {  
  const naheX = x > zielX - 30 && x < zielX + 30  
  const naheY = y > zielY - 30 && y < zielY + 30  
  if (naheX && naheY) {  
    fangen()  
  }  
}
```

Nah in x *und* nah in y, dann hast du den Stern erwischt.

Beim Fangen: Punkt und neuer Ort

```
function fangen() {  
  punkte = punkte + 1  
  document.getElementById("punkte").innerHTML = "Punkte: " + punkte  
  
  zielX = Math.floor(Math.random() * 400)  
  zielY = Math.floor(Math.random() * 300)  
  document.getElementById("stern").style.left = zielX + "px"  
  document.getElementById("stern").style.top = zielY + "px"  
}
```

Ein Punkt mehr, und der Stern springt an eine zufällige neue Stelle.

Alles zusammenspielen

Damit geprüft wird, ruft bewege am Ende einfach pruefe auf:

```
function bewege(event) {  
  // ... Figur bewegen wie vorher ...  
  pruefe()  
}
```

Bewegen, prüfen, fangen: drei kleine Funktionen, die zusammen ein Spiel ergeben.



Mach es spannend: ein Timer

Erinnerst du dich an Modul 6? Mit einem Timer wird aus dem Sammeln ein echtes Wettrennen:

- ein 30-Sekunden-Countdown mit `setInterval`
- wie viele Sterne schaffst du, bevor die Zeit abläuft?
- bei 0 stoppst du mit `clearInterval` und zeigst die Punkte

Übung: dein Fang-Spiel



PROJEKT

ca. 15 Minuten

- Bring das Fang-Spiel zum Laufen
- Wie viele Sterne schaffst du?

★ **Bonus:** Baue mit einem Timer aus Modul 6 einen 30-Sekunden-Countdown ein.

★ Zeig dein Fang-Spiel!

Punkte: 7



Held

Pfeiltasten drücken und die Sterne einsammeln.



Projekt

Das Ausweich-Spiel



Jetzt öffnen: [7b-ausweich-spiel](#)

in VS Code – oder auf der Kursseite

Wie es geht

- deine Figur steht unten und geht nach links und rechts
- von oben fällt ein Hindernis herunter
- weich ihm aus! Erwischt es dich, ist das Spiel vorbei
- für jedes Hindernis, dem du entkommst, gibt es einen Punkt

Alles bekannt: Pfeiltasten (Modul 7) und ein Timer (Modul 6).



So ist deine Vorlage aufgebaut

```
<h2 id="punkte">Punkte: 0</h2>
<div id="spielfeld">
  <div id="spieler"></div>
  <div id="hindernis"></div>
</div>
<button onclick="start()">Start</button>
```

Ein **spieler** unten, ein **hindernis** oben. Beide haben `position: absolute`.

Links und rechts

Beim Ausweichen brauchst du nur zwei Richtungen:

```
let x = 200

document.addEventListener("keydown", bewege)

function bewege(event) {
  if (event.key === "ArrowLeft") x = x - 20
  // ... und ArrowRight für x + 20
  document.getElementById("spieler").style.left = x + "px"
}
```

Der Timer startet

```
let hindernisX = 100
let hindernisY = 0
let punkte = 0
let timer

function start() {
  timer = setInterval(fallen, 50)
}
```

„Start“ lässt das Hindernis alle 50 ms ein Stück fallen.

⚡ Das Hindernis fällt

```
function fallen() {  
    hindernisY = hindernisY + 5  
    if (hindernisY > 340) { // unten? neu von oben  
        hindernisY = 0  
        hindernisX = Math.floor(Math.random() * 400)  
        punkte = punkte + 1  
        document.getElementById("punkte").innerHTML = "Punkte: " +  
        punkte  
    }  
    document.getElementById("hindernis").style.top = hindernisY + "px"  
    document.getElementById("hindernis").style.left = hindernisX +  
        "px"  
    pruefe()  
}
```




Getroffen?

Nach jedem Schritt prüfen wir, ob das Hindernis unten den Spieler berührt:

```
function pruefe() {  
  const naheX = hindernisX > x - 30 && hindernisX < x + 30  
  const untenY = hindernisY > 300  
  if (naheX && untenY) {  
    clearInterval(timer)  
    document.getElementById("punkte").innerHTML = "Getroffen!  
Punkte: " + punkte  
  }  
}
```

`clearInterval` stoppt den Timer, das Spiel ist vorbei.

Übung: dein Ausweich-Spiel



PROJEKT

ca. 15 Minuten

- Bring das Spiel zum Laufen und weiche aus
- Mach es schwerer: lass das Hindernis schneller fallen

★ **Bonus:** Lass zwei Hindernisse gleichzeitig fallen.

Zeig dein Ausweich-Spiel!

Punkte: 12



Du

Weich dem fallenden Hindernis aus.

✓ Das kannst du jetzt

- mit `addEventListener` auf die **Tastatur** hören
- mit `event.key` erkennen, welche Taste gedrückt wurde
- eine Figur über `x`, `y` und `style.left/top` **bewegen**
- prüfen, ob sich zwei Dinge **berühren**

Bewegung, Zufall, Timer, Entscheidungen: du hast alles für echte Spiele!

☰ Neue Befehle auf einen Blick

addEventListener("keydown", fn)	=	auf Tasten hören
event.key	=	welche Taste
style.left = x + "px"	=	von links verschieben
style.top = y + "px"	=	von oben verschieben

Mehr Ideen für Bewegungs-Spiele

Mit Bewegung kannst du jetzt viele Spiele bauen:

- ein **Labyrinth**: finde den Weg zum Ziel
- ein **Ausweich-Spiel**: weich fallenden Dingen aus
- **zwei Spieler**: einer mit den Pfeiltasten, einer mit w a s d

Alles nur mit dem, was du schon kannst.



Jetzt bist du dran

Du hast alle Bausteine für echte Spiele:

Variablen, Funktionen, Entscheidungen, Zufall, Schleifen, Timer und Steuerung.

Kombiniere sie zu deinem eigenen Spiel. Der Rest ist deine Fantasie.